# CS 225

**Data Structures**

**Volodymyr Kindratenko**

*Slides courtesy of Wade Fagen-Ulmschneider*

# Reference Variable

A reference is an ***alias*** for an existing object, effectively another name bound to the same memory location.

**Key properties:**

- Must be initialized when declared.
- Cannot be reassigned to refer to a different object.
- No null references.

**Syntax:**

```
T& name = object;
```

# Pointers and References

| CONCEPT | SYNTAX | DESCRIPTION |
|---|---|---|
| Object Instance | Sphere s1; | Creates an instance of Sphere |
| Reference Variable | Sphere& r1 = s1; | Alias for s1, cannot be reseated |
| Pointer Variable | Sphere* p1 = &s1; | Stores address of a Sphere object |

# heap-puzzle3.cpp

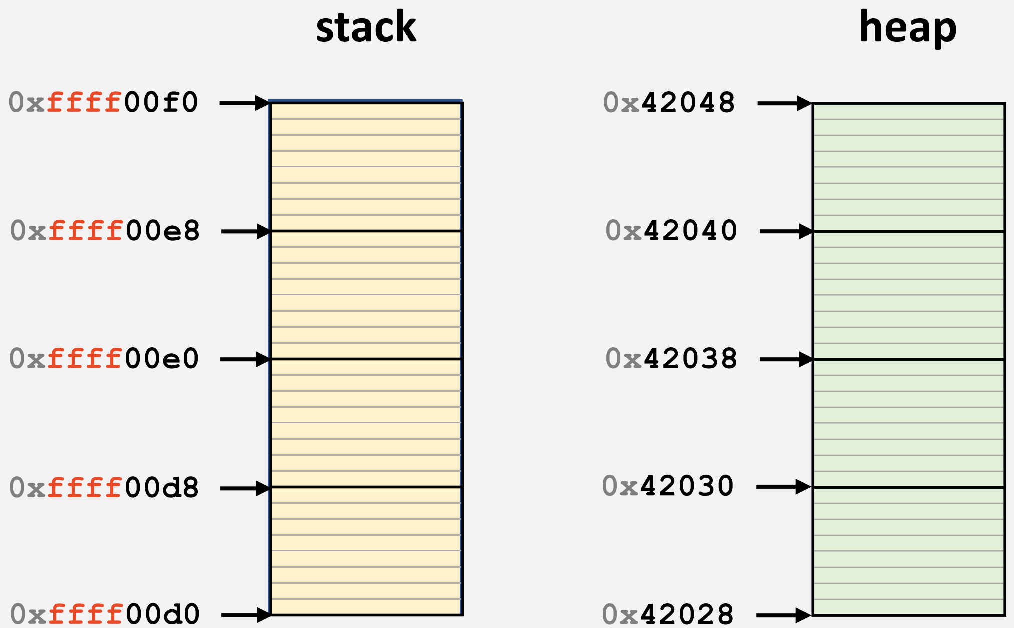
```cpp
#include <iostream>
using namespace std;

int main() {
  int *x = new int;
  int &y = *x;

  y = 4;

  cout << &x << endl;
  cout << x << endl;
  cout << *x << endl;

  cout << &y << endl;
  cout << y << endl;
  cout << *y << endl;
}
```



# joinSpheres-byValue.cpp

```
/*
 * Creates a new sphere that contains the exact volume
 * of the volume of the two input spheres.
 */
Sphere joinSpheres(Sphere s1, Sphere s2) {
  double totalVolume = s1.getVolume() + s2.getVolume();

  double newRadius = std::pow(
    (3.0 * totalVolume) / (4.0 * 3.141592654),
    1.0/3.0
  );

  Sphere result(newRadius);

  return result;
}
```

```
int main() {
  Sphere *s1 = new Sphere(4);
  Sphere *s2 = new Sphere(5);

  Sphere s3 = joinSpheres(*s1, *s2);

  return 0;
}
```

# joinSpheres-byPointer.cpp

```
/*
 * Creates a new sphere that contains the exact volume
 * of the volume of the two input spheres.
 */
Sphere joinSpheres(Sphere *s1, Sphere *s2) {
  double totalVolume = s1->getVolume() + s2->getVolume();

  double newRadius = std::pow(
    (3.0 * totalVolume) / (4.0 * 3.141592654),
    1.0/3.0
  );

  Sphere result(newRadius);

  return result;
}
```

```
int main() {
  Sphere *s1 = new Sphere(4);
  Sphere *s2 = new Sphere(5);

  Sphere s3 = joinSpheres(s1, s2);

  return 0;
}
```

# joinSpheres-byReference.cpp

```cpp
/*
 * Creates a new sphere that contains the exact volume
 * of the volume of the two input spheres.
 */
Sphere joinSpheres(Sphere &s1, Sphere &s2) {
  double totalVolume = s1.getVolume() + s2.getVolume();

  double newRadius = std::pow(
    (3.0 * totalVolume) / (4.0 * 3.141592654),
    1.0/3.0
  );

  Sphere result(newRadius);

  return result;
}
```

```cpp
int main() {
  Sphere *s1 = new Sphere(4);
  Sphere *s2 = new Sphere(5);

  Sphere s3 = joinSpheres(*s1, *s2);

  return 0;
}
```

# Parameter Passing Properties

| | **By Value**<br>`void foo(Sphere a) { … }` | **By Pointer**<br>`void foo(Sphere *a) { … }` | **By Reference**<br>`void foo(Sphere &a) { … }` |
|---|---|---|---|
| Exactly what is copied when the function is invoked? | | | |
| Does modification of the passed in object modify the caller's object? | | | |
| Is there always a valid object passed in to the function? | | | |
| Speed | | | |
| Programming Safety | | | |

# const Parameters

In function parameters, `const` promises the caller that the value passed in will not be modified.

This is enforced by the compiler.

## joinSpheres-byValue-const.cpp

```
/*
 * Creates a new sphere that contains the exact volume
 * of the volume of the two input spheres.
 */
Sphere joinSpheres(const Sphere s1, const Sphere s2) {
  double totalVolume = s1.getVolume() + s2.getVolume();

  double newRadius = std::pow(
    (3.0 * totalVolume) / (4.0 * 3.141592654),
    1.0/3.0
  );

  Sphere result(newRadius);

  return result;
}
```

```
int main() {
  Sphere *s1 = new Sphere(4);
  Sphere *s2 = new Sphere(5);

  Sphere s3 = joinSpheres(*s1, *s2);

  delete s1; s1 = nullptr;
  delete s2; s2 = nullptr;

  return 0;
}
```

# joinSpheres-byPointer-const.cpp

```
/*
 * Creates a new sphere that contains the exact volume
 * of the volume of the two input spheres.
 */
Sphere joinSpheres(Sphere const *s1, Sphere const *s2) {
  double totalVolume = s1->getVolume() + s2->getVolume();

  double newRadius = std::pow(
    (3.0 * totalVolume) / (4.0 * 3.141592654),
    1.0/3.0
  );

  Sphere result(newRadius);

  return result;
}
```

```
int main() {
  Sphere *s1 = new Sphere(4);
  Sphere *s2 = new Sphere(5);

  Sphere s3 = joinSpheres(s1, s2);

  delete s1; s1 = nullptr;
  delete s2; s2 = nullptr;

  return 0;
}
```

## joinSpheres-byReference-const.cpp

```cpp
/*
 * Creates a new sphere that contains the exact volume
 * of the volume of the two input spheres.
 */
Sphere joinSpheres(const Sphere &s1, const Sphere &s2) {
  double totalVolume = s1.getVolume() + s2.getVolume();

  double newRadius = std::pow(
    (3.0 * totalVolume) / (4.0 * 3.141592654),
    1.0/3.0
  );

  Sphere result(newRadius);

  return result;
}
```

```cpp
int main() {
  Sphere *s1 = new Sphere(4);
  Sphere *s2 = new Sphere(5);

  Sphere s3 = joinSpheres(*s1, *s2);

  delete s1; s1 = nullptr;
  delete s2; s2 = nullptr;

  return 0;
}
```

```
[waf@linux-a2 5]$ clang++ -fno-elide-constructors -std=c++11 -stdlib=libc++ -O0
joinSpheres-byValue-const.cpp sphere.cpp
joinSpheres-byValue-const.cpp:16:24: error: member function 'getVolume' not
      viable: 'this' argument has type 'const cs225::Sphere', but function is
      not marked const
  double totalVolume = s1.getVolume() + s2.getVolume();
                       ^~
./sphere.h:12:12: note: 'getVolume' declared here
    double getVolume() ;
           ^
joinSpheres-byValue-const.cpp:16:41: error: member function 'getVolume' not
      viable: 'this' argument has type 'const cs225::Sphere', but function is
      not marked const
  double totalVolume = s1.getVolume() + s2.getVolume();
                                        ^~
./sphere.h:12:12: note: 'getVolume' declared here
    double getVolume() ;
           ^
2 errors generated.
```

# const functions in classes

Functions in classes are **constant** if they do not modify the state of the class

## sphere.h

```cpp
#ifndef SPHERE_H
#define SPHERE_H

namespace cs225 {
  class Sphere {
    public:
      Sphere();
      Sphere(double r);

      double getRadius() const;
      double getVolume() const;

      void setRadius(double r);

    private:
      double r_;

  };
}

#endif
```

## sphere.cpp

```cpp
#include "sphere.h"

namespace cs225 {
  Sphere::Sphere() : Sphere(1) { }

  Sphere::Sphere(double r) {
     r_ = r;
  }

  double Sphere::getRadius() const {
    return r_;
  }

  double Sphere::getVolume() const {
    return (4 * r_ * r_ * r_ *
                3.14159265) / 3.0;
  }

  void setRadius(double r) {
     r_ = r;
  }
}
```

# Copy Constructor

It is a special constructor called when a copy of an object is made

Syntax: **`Sphere(const Sphere &)`**

# Copy Constructor

**Automatic Copy Constructor**

- Exists only if no copy constructor is defined
- Copies all values in the class
- It is a shallow copy, because it does not copy objects pointed by pointers

**Custom Copy Constructor**

- Define custom constructor if we need to copy something from pointers sphere.h

```cpp
#ifndef SPHERE_H
#define SPHERE_H

namespace cs225 {
  class Sphere {
    public:
      Sphere();
      Sphere(double r);
      Sphere(const Sphere &s);

      double getRadius() const;
      double getVolume() const;

      void setRadius(double r);

    private:
      double r_;

  };
}

#endif
```

sphere.cpp

```cpp
#include "sphere.h"
#include <iostream>

using namespace std;

namespace cs225 {
  Sphere::Sphere() : Sphere(1) {
      cout << "Default ctor" << endl;
  }

  Sphere::Sphere(double r) {
      cout << "1-param ctor" << endl;
      r_ = r;
  }

  Sphere::Sphere(const Sphere &s) {
      r_ = s.r_;
      cout << "Copy ctor" << endl;
  }


...
  // ...
```

# joinSpheres-byValue-const.cpp

```
/*
 * Creates a new sphere that contains the exact volume
 * of the volume of the two input spheres.
 */
Sphere joinSpheres(const Sphere s1, const Sphere s2) {
  double totalVolume = s1.getVolume() + s2.getVolume();

  double newRadius = std::pow(
    (3.0 * totalVolume) / (4.0 * 3.141592654),
    1.0/3.0
  );

  Sphere result(newRadius);

  return result;
}
```

```
int main() {
  Sphere *s1 = new Sphere(4);
  Sphere *s2 = new Sphere(5);

  Sphere s3 = joinSpheres(*s1, *s2);

  delete s1; s1 = nullptr;
  delete s2; s2 = nullptr;

  return 0;
}
```

# joinSpheres-byPointer-const.cpp

```
/*
 * Creates a new sphere that contains the exact volume
 * of the volume of the two input spheres.
 */
Sphere joinSpheres(Sphere const *s1, Sphere const *s2) {
  double totalVolume = s1->getVolume() + s2->getVolume();

  double newRadius = std::pow(
    (3.0 * totalVolume) / (4.0 * 3.141592654),
    1.0/3.0
  );

  Sphere result(newRadius);

  return result;
}
```

```
int main() {
  Sphere *s1 = new Sphere(4);
  Sphere *s2 = new Sphere(5);

  Sphere s3 = joinSpheres(s1, s2);

  delete s1; s1 = nullptr;
  delete s2; s2 = nullptr;

  return 0;
}
```

# joinSpheres-byReference-const.cpp

```
/*
 * Creates a new sphere that contains the exact volume
 * of the volume of the two input spheres.
 */
Sphere joinSpheres(const Sphere &s1, const Sphere &s2) {
  double totalVolume = s1.getVolume() + s2.getVolume();

  double newRadius = std::pow(
    (3.0 * totalVolume) / (4.0 * 3.141592654),
    1.0/3.0
  );

  Sphere result(newRadius);

  return result;
}
```

```
int main() {
  Sphere *s1 = new Sphere(4);
  Sphere *s2 = new Sphere(5);

  Sphere s3 = joinSpheres(*s1, *s2);

  delete s1; s1 = nullptr;
  delete s2; s2 = nullptr;

  return 0;
}
```

# Calls to constructors

| | By Value `void foo(Sphere a) { … }` | By Pointer `void foo(Sphere *a) { … }` | By Reference `void foo(Sphere &a) { … }` |
|---|---|---|---|
| `Sphere::Sphere()` | | | |
| `Sphere::Sphere(double)` | | | |
| `Sphere::Sphere(const Sphere&)` | | | |

# CS 225 – Things To Be Doing

**Quiz 1 is on Thursday**

At 19:00 in your discussion

More Info: see Blackboard

**Work on MP1**

Due: Monday, March 2nd

**Attend lab/discussion and complete lab_intro**

Due: Sunday, January 25th